

Итоговый отчет о выполненной работе

Исследование стоимости сопряжений на циклических кривых в EVM и перспектив дешевого folding

MNT4-753, MNT6-753 и исследовательская конструкция lollipop-305

Содержание

1	Постановка задачи и практическая мотивация	2
1.1	Статус результата	3
2	Теоретическая база вычисления сопряжения	3
2.1	Цикл Миллера и финальная экспонента	3
2.2	Удаление знаменателя	4
2.3	Проверка финальной экспоненты через свидетельство	4
3	Оптимизированная 3-limb арифметика MNT4-753	5
3.1	Представление данных	5
3.2	Почему выбран Montgomery/CIOS	5
3.3	Нижняя оценка для 3-limb умножения	5
3.4	Башня расширений и Karatsuba	6
3.5	Подготовленные разреженные линии	7
4	MNT4: от наивного Tate pairing к оптимизированному пути	7
4.1	Наивная точка отсчета	7
4.2	Оптимизационная лестница	8
4.3	Формальная нижняя оценка стоимости Miller core	8
5	MNT4 direct residue verifier по ePrint 2024/640	9
5.1	API и граница доверия	9
5.2	Результаты MNT4	9
6	Проверка цикла Миллера через полиномиальные отношения	10
6.1	Математическая идея	10
6.2	KZG over BN254	10
6.3	Merkle/FRI	11
7	MNT6-753 и цикл MNT4/MNT6	12
7.1	Почему нужна вторая кривая	12
7.2	MNT6 arithmetic и residue verifier	12
7.3	Cycle-native accounting	13
8	Исследовательское направление lollipop-305	14
8.1	Идея	14
8.2	Кривые конструкции	14
8.3	Оптимизированная 2-limb арифметика	14
8.4	Почему для третьей кривой нужны два аккумулятора	14

8.5	Результаты lollipop-305	15
9	Сводные результаты и trade-off	16
9.1	Три формы стоимости	16
9.2	Экономическая интерпретация	17
10	Итоговые выводы	17
11	Воспроизводимость	18

1 Постановка задачи и практическая мотивация

Работа посвящена проверяемым вычислениям на эллиптических кривых в EVM. Практическая мотивация связана с рекурсивными доказательствами. Если требуется построить цепочку доказательств, недостаточно один раз проверить отдельное утверждение. Нужно уметь эффективно доказывать корректность предыдущего шага внутри следующего шага. На этой идее строятся folding-схемы и рекурсивные SNARK-системы.

В существующих реализациях, включая Sonobe/CycleFold, заметная часть стоимости может возникать не из-за пользовательского утверждения, а из-за переноса арифметики одной кривой в поле другой кривой. Такая арифметика называется *non-native*. Элемент большого поля раскладывается на машинные слова (*limb*-ы), а каждое умножение требует ограничений на частичные произведения, переносы и редукцию. В качестве практического ориентира в работе используется порядок около 9 млн constraints для Sonobe/CycleFold-like decider. Этот показатель не является прямым apples-to-apples сравнением с каждым отдельным фрагментом настоящей работы, но хорошо показывает масштаб проблемы.

Для Ethereum задача дополнительно осложняется архитектурой EVM:

- EVM работает с 256-битными словами;
- для BN254 существуют предкомпилированные операции;
- для MNT4-753 и MNT6-753 аналогичных предкомпилированных операций нет;
- элементы MNT-полей имеют размер около 753 бит и требуют трех слов EVM.

Поэтому были исследованы два фундаментальных пути:

1. выполнять арифметику сопряжения непосредственно on-chain и платить за нее gas;
2. переносить вычисление off-chain и платить за доказательство, constraints, calldata и время работы доказчика.

Цель работы состоит не в том, чтобы заранее выбрать один путь, а в том, чтобы реализовать и измерить основные варианты, определить границу применимости и проверить, дает ли цикл MNT4/MNT6 или конструкция lollipop более дешевую основу для будущего folding.

1.1 Статус результата

Практическая исследовательская часть завершена. Реализованы:

- оптимизированная 3-limb арифметика MNT4-753 и MNT6-753;
- полное on-chain MNT4-сопряжение как reference baseline;
- наивная модель стоимости Tate pairing;
- direct residue verifier по статье *On Proving Pairings*;
- KZG- и Merkle/FRI-направления для проверки трассы цикла Миллера;
- предварительный MNT4/MNT6 cycle-native accounting;
- исследовательская реализация lollipop-305 с 2-limb арифметикой.

Главный результат имеет граничный характер: без новых системных возможностей нельзя одновременно получить малую стоимость on-chain и малую стоимость off-chain proof-слоя простым переносом существующей арифметики между слоями.

2 Теоретическая база вычисления сопряжения

2.1 Цикл Миллера и финальная экспонента

Для reduced Tate pairing используется вычисление функции Миллера и финальная экспонента:

$$e(P, Q) = f_{r,P}(Q)^{(q^k-1)/r}.$$

Здесь r — порядок подгруппы, q — характеристика базового поля, а k — степень вложения. Алгоритм Миллера строит $f_{r,P}$ через последовательность удвоений и сложений. Для каждого шага обновляется аккумулятор:

$$f \leftarrow f^2 \cdot \ell_{\text{dbl}}(Q),$$

а для ненулевой цифры signed-представления дополнительно:

$$f \leftarrow f \cdot \ell_{\text{add}}(Q).$$

Для уравнения сопряжений

$$e(P, Q) = e(R, S)$$

удобно проверять эквивалентную форму

$$e(P, Q) \cdot e(-R, S) = 1.$$

Обе пары можно обрабатывать общим аккумулятором:

$$f \leftarrow f^2 \cdot \ell_{Q,i}(P) \cdot \ell_{S,i}(-R).$$

Главное преимущество общего аккумулятора: возведение в квадрат выполняется один раз на раунд, а не отдельно для каждой пары.

2.2 Удаление знаменателя

В общей формуле функции Миллера возникает отношение линии к вертикальной линии:

$$g_{A,B}(X) = \frac{\ell_{A,B}(X)}{v_{A+B}(X)}.$$

Для MNT4-753 знаменатель после выбора групп лежит в собственном подполе. Финальная экспонента уничтожает этот множитель. Поэтому в вычислительном пути можно использовать только числитель линии. В полном тексте диплома приведено формальное доказательство корректности такого удаления.

Практический смысл оптимизации существенен: на каждом шаге не требуется вычислять дополнительное обращение или вести второй аккумулятор знаменателя. Это свойство не является автоматически верным для любой кривой. Ниже будет показано, что для третьей кривой lollipop-305 оно не выполняется в нужной форме, и это напрямую увеличивает стоимость.

2.3 Проверка финальной экспоненты через свидетельство

Статья *On Proving Pairings* [1] позволяет заменить полную финальную экспоненту проверкой отношения. Для уравнения сопряжений пусть F — результат объединенного цикла Миллера. Требуется проверить:

$$F^{(q^k-1)/r} = 1.$$

При выполнении условий теоремы статьи это равносильно существованию элемента c , для которого:

$$F = c^r.$$

Вместо длинной цепочки возведений в степень контракт проверяет эквивалентное отношение:

$$F \cdot c^{-r} = 1.$$

Часть степени c^{-r} встраивается в тот же signed double-and-add цикл, а возведение в степень q выполняется отображением Фробениуса.

Для MNT4 и MNT6 знаки различаются:

$$r_{\text{MNT4}} = q_{\text{MNT4}} + N, \quad r_{\text{MNT6}} = q_{\text{MNT6}} - N.$$

Следовательно:

$$c^{-r_{\text{MNT4}}} = c^{-N-q}, \quad c^{-r_{\text{MNT6}}} = c^{N-q}.$$

Это различие было учтено в финальной реализации MNT6.

3 Оптимизированная 3-limb арифметика MNT4-753

3.1 Представление данных

Элемент базового поля MNT4-753 не помещается в одно слово EVM и хранится как три 256-битных слова:

$$x = x_0 + 2^{256}x_1 + 2^{512}x_2.$$

Для длинных цепочек умножений используется Montgomery-представление. Пусть

$$B = 2^{256}, \quad R = B^3 = 2^{768}.$$

Тогда:

$$\tilde{x} = xR \bmod p,$$

$$\text{MontMul}(\tilde{x}, \tilde{y}) = \tilde{x}\tilde{y}R^{-1} \bmod p = xyR \bmod p.$$

Результат остается в Montgomery-домене. Это важно для цикла Миллера: вход в домен выполняется один раз, после чего тысячи операций не требуют повторного преобразования.

3.2 Почему выбран Montgomery/CIOS

Были реализованы и измерены несколько вариантов:

- Montgomery/CIOS;
- Barrett-редукция;
- FIOS;
- Comba/SOS;
- square-specialized Comba;
- branchless reduction;
- lazy reduction variants.

CIOS (*Coarsely Integrated Operand Scanning*) объединяет накопление частичных произведений и Montgomery-редукцию. Для EVM это выгоднее, чем материализация полного 6-словного произведения в памяти.

3.3 Нижняя оценка для 3-limb умножения

Для произведения двух трехсловных чисел требуется не менее девяти попарных произведений:

$$ab = \sum_{i=0}^2 \sum_{j=0}^2 a_i b_j 2^{256(i+j)}.$$

Montgomery-редукция выполняет три шага. На каждом шаге добавляется произведение корректирующего коэффициента на трехсловный модуль, то есть еще три произведе-

Таблица 1: Сравнение базовых 3-limb операций

Вариант	Операция	Gas/op	Вывод
Montgomery/CIOS	F_q mul	2,959	основной путь
Montgomery/CIOS	F_q square	2,947	основной путь
Barrett	F_q mul	46,998	существенно дороже
Barrett	F_q square	47,172	существенно дороже
FIOS	F_q mul	18,509	дороже CIOS
FIOS	F_q square	18,543	дороже CIOS
Comba/SOS	F_q mul	18,301	проигрывает из-за memory traffic
Square-Comba	F_q square	18,420	лучше общего Comba, но хуже CIOS

ния на шаг. Получаем:

$$9 \text{ произведений для } ab + 9 \text{ произведений для REDC} = 18$$

полных 512-битных limb-произведений.

В EVM нет инструкции MULHI. Для восстановления старшей половины 512-битного произведения используется связка:

```
lo := mul(u, v)
mm := mulmod(u, v, not(0))
hi := sub(sub(mm, lo), lt(mm, lo))
```

Дополнительно нужны переносы, нормализация и условное вычитание модуля. Поэтому значение около 2,959 gas/op является не случайной неэффективностью, а близким к практически достижимому уровню для выбранного представления.

3.4 Башня расширений и Karatsuba

Для MNT4 используется башня:

$$\mathbb{F}_{q^2} = \mathbb{F}_q[u]/(u^2 - 13), \quad \mathbb{F}_{q^4} = \mathbb{F}_{q^2}[v]/(v^2 - u).$$

Для квадратичного расширения Karatsuba заменяет четыре умножения тремя:

$$\begin{aligned} t_0 &= a_0 b_0, \\ t_1 &= a_1 b_1, \\ t_2 &= (a_0 + a_1)(b_0 + b_1), \\ c_0 &= t_0 + 13t_1, \\ c_1 &= t_2 - t_0 - t_1. \end{aligned}$$

Та же идея применяется поверх $\mathbb{F}_{q^2}$ для $\mathbb{F}_{q^4}$.

Умножения на нерезиденты реализованы отдельными формулами. Например:

$$(x_0 + x_1 u)u = 13x_1 + x_0 u.$$

Это значительно дешевле общего умножения.

Таблица 2: Стоимость операций расширений MNT4-753

Операция	Gas/op
$\mathbb{F}_q$ mul	2,959
$\mathbb{F}_q$ square	2,947
$\mathbb{F}_{q^2}$ mul	11,877
$\mathbb{F}_{q^2}$ square	10,592
$\mathbb{F}_{q^4}$ mul	42,764
$\mathbb{F}_{q^4}$ square	36,606

Таблица 3: Эффект специализированных умножений на нерезиденты

Операция	Общий путь	Специализированный путь	Выигрыш
умножение на 13 в $\mathbb{F}_q$	2,961	1,481	2.0×
умножение на u в $\mathbb{F}_{q^2}$	12,587	1,731	7.3×
умножение на v в $\mathbb{F}_{q^4}$	42,253	1,859	22.7×

3.5 Подготовленные разреженные линии

После подстановки точки линия Миллера имеет специальную структуру. Ее не нужно представлять как произвольный плотный элемент $\mathbb{F}_{q^4}$. В проекте реализованы:

- prepared fixed- Q cache;
- sparse-формат коэффициентов;
- специализированное умножение аккумулятора на линию;
- pointer/packed memory API;
- потоковое чтение code-shards через EXTCODECOPY;
- экспериментальное слияние $f \leftarrow f^2 \ell(P)$ без лишнего промежуточного объекта.

Последний агрессивный fused-вариант корректен, но дает лишь около 20 тыс. gas экономии на полном single-pairing пути. Это показывает, что после оптимизации памяти доминирует уже не копирование данных, а собственно арифметика расширений.

4 MNT4: от наивного Tate pairing к оптимизированному пути

4.1 Наивная точка отсчета

Для честного сравнения построена наивная модель Tate pairing. Она намеренно не использует:

- короткий Ate loop;
- prepared lines;
- sparse multiplication;
- Yul hot path;

- Frobenius-декомпозицию финальной экспоненты.

Полный неоптимизированный вызов превышает практические лимиты EVM, поэтому измерены корректные микроблоки и построена строгая нижняя экстраполяция:

$$\begin{aligned}\text{Miller} &= 752 \cdot 973,261 + 372 \cdot 486,819 \\ &= 912,988,940 \text{ gas}, \\ \text{FE} &= 2259 \cdot 486,899 + 1100 \cdot 486,819 \\ &= 1,635,405,741 \text{ gas}.\end{aligned}$$

Итого:

$$2,548,394,681 \text{ gas}.$$

Это нижняя оценка: она не включает построение линий, проверки входов и часть накладных расходов.

4.2 Оптимизационная лестница

Таблица 4: Интегральная лестница MNT4

Шаг	Реализация	Gas	Снижение к предыдущему шагу
0	Наивная Tate cost model	2,548,394,681	—
1	Полный optimized fixed- Q on-chain путь	258,753,182	89.85%
2	Fixed- Q prepared sparse blob	79,588,799	69.24%
3	Fixed- Q prepared sparse code-shards	79,586,596	0.003%

Оптимизации дают более чем десятикратное снижение относительно наивного пути. Однако даже prepared-вариант остается дороже обычного лимита блока Ethereum.

4.3 Формальная нижняя оценка стоимости Miller core

Для двухпарного уравнения:

$$e(P, Q)e(-R, S) = 1$$

используем:

$$L = 376, \quad H = 123,$$

где L — число удвоений, H — число ненулевых addition-шагов. Минимально нужны:

- L возведений аккумулятора в квадрат;
- $2L$ применений doubling-линий;
- $2H$ применений addition-линий.

Грубая нижняя оценка через измеренные $\mathbb{F}_{q^4}$ -операции:

$$376 \cdot 36,606 = 13,763,856,$$

$$2 \cdot 376 \cdot 42,764 = 32,157,728,$$

$$2 \cdot 123 \cdot 42,764 = 10,499,864.$$

Суммарно:

$$56,421,448 \text{ gas}.$$

Реальный код дополнительно загружает sparse-коэффициенты, вычисляет значения линий, работает с памятью и обрабатывает signed loop. Поэтому measured Miller core порядка 87.7 млн gas согласуется с формальной моделью.

5 MNT4 direct residue verifier по ePrint 2024/640

5.1 API и граница доверия

В основном fixed-shards режиме контракт проверяет:

$$e(P, Q)e(-R, S) = 1,$$

где $Q, S \in G_2$ фиксированы при развертывании, а $P, R \in G_1$ передаются пользователем. Prepared-кэш линий хранится в runtime-коде data-контрактов. Адреса shards фиксируются в конструкторе. Пользователь не может заменить их при вызове.

Это доверенная предрегистрация fixed-cache: контракт защищен от подмены кэша во время вызова, но не доказывает заново, что развертыватель построил линии правильно. Для произвольной меняющейся точки Q нужен отдельный proof корректности кэша или прямой пересчет.

5.2 Результаты MNT4

Таблица 5: Direct residue verifier MNT4

Сценарий	Что выполняется	Gas
Miller core	цикл Миллера без финальной экспоненты	87,747,219
Full equation	Miller core + обычная FE	115,997,313
Residue equation	Miller core + c -проверка вместо полной FE	93,879,746
Fixed-shards residue equation с проверкой G_1	основной fixed-cache API	93,734,789

Вклад обычной FE:

$$115,997,313 - 87,747,219 = 28,250,094 \text{ gas}.$$

Вклад residue-проверки:

$$93,879,746 - 87,747,219 = 6,132,527 \text{ gas.}$$

Экономия:

$$22,117,567 \text{ gas.}$$

Оптимизация финальной экспоненты работает, но не устраняет главный расход: цикл Миллера.

6 Проверка цикла Миллера через полиномиальные отношения

6.1 Математическая идея

Следующий шаг статьи [1] — не исполнять все умножения расширенного поля on-chain, а проверить их как полиномиальные отношения. Пусть:

$$\mathbb{F}_{q^4} \simeq \mathbb{F}_q[X]/(p(X)).$$

Для умножения $a \cdot b = c$ существует частный многочлен $Q(X)$:

$$a(X)b(X) - c(X) = Q(X)p(X).$$

Verifier выбирает случайную точку α и проверяет:

$$a(\alpha)b(\alpha) - c(\alpha) = Q(\alpha)p(\alpha).$$

Для набора отношений используется challenge β :

$$\sum_i \beta^i R_i(X) = Q(X)Z_H(X).$$

Для цикла Миллера R_i кодируют реальные переходы:

$$f_{i+1} = f_i^2 \ell_{Q,i}(P) \ell_{S,i}(-R).$$

Чтобы prover не подменил значения после получения α , нужны commitments и доказательство открытия.

6.2 KZG over BN254

KZG дает короткое доказательство открытия и дешевую on-chain проверку благодаря BN254 precompile. В проекте реализован opening layer:

$$C_j = \text{Commit}(w_j), \quad C_Q = \text{Commit}(Q).$$

Measured стоимость одной on-chain opening-проверки составляет около:

133,039 gas.

Проблема переносится внутрь circuit: MNT4-арифметика становится non-native относительно BN254.

Таблица 6: Оценка BN254 non-native constraints

Операция	Constraints
MNT4 $\mathbb{F}_q$ mul как non-native	3,692
MNT4 $\mathbb{F}_{q^2}$ mul как non-native	11,300
Модель MNT4 $\mathbb{F}_{q^4}$ mul как non-native	63,436
Приближенная sparse Miller relation	63,309,128

То есть KZG уменьшает gas verifier-а, но возвращает исходную проблему роста constraints.

6.3 Merkle/FRI

В альтернативном направлении prover коммитится к таблицам через Merkle root. Verifier проверяет Merkle-пути открытых строк. FRI (*Fast Reed–Solomon Interactive Oracle Proof of Proximity*) доказывает, что открываемые таблицы согласуются с многочленами ограниченной степени. Fiat–Shamir-преобразование делает схему неинтерактивной: challenges детерминированно вычисляются из transcript.

В проекте реализованы:

- Merkle opening layer;
- исполняемый архивный DEEP-FRI отрицательный эксперимент;
- актуальная ordinary-FRI cost model для нативного MNT4-поля.

Полноценная production-замена Miller loop не заявляется: результат используется как строгая stop/go модель стоимости.

Для MNT4 один элемент поля занимает:

$$3 \cdot 32 = 96 \text{ байт.}$$

Умеренная модель открытий:

$$4096 \cdot 96 + 128 \cdot 16 \cdot 32 + 16 \cdot 32 = 459,264 \text{ байт.}$$

Актуальная ordinary-FRI модель для 128-битного профиля:

Вывод: Merkle/FRI может быть дешевле Article640 fixed-shards verifier, но не достигает целевого уровня ниже 60 млн gas. Стоимость переносится в calldata, Merkle hashing и раскрытия таблиц.

Таблица 7: Ordinary-FRI cost model

Показатель	Значение
Число запросов	576
Оценка soundness	135.76 бит
FRI schedule	[2, 2, 2, 4]
Размер доказательства	2,072,224 байт
Строгая нижняя оценка	51,359,352 gas
Ожидаемая модельная оценка	78,340,624 gas

7 MNT6-753 и цикл MNT4/MNT6

7.1 Почему нужна вторая кривая

MNT4-753 и MNT6-753 образуют цикл полей:

$$\mathbb{F}_r(\text{MNT4}) = \mathbb{F}_q(\text{MNT6}),$$

$$\mathbb{F}_r(\text{MNT6}) = \mathbb{F}_q(\text{MNT4}).$$

Для будущего folding это принципиально: арифметику одной стороны можно выражать ближе к родному полю другой стороны, избегая наиболее грубой BN254 non-native эмуляции.

7.2 MNT6 arithmetic и residue verifier

Для MNT6 реализована башня:

$$\mathbb{F}_q \longrightarrow \mathbb{F}_{q^3} \longrightarrow \mathbb{F}_{q^6}.$$

Реализованы packed/pointer API, scratch arena, prepared cache, потоковое чтение code-shards, fused line multiplication и Frobenius/w0 FE.

Таблица 8: Базовые операции MNT6

Операция	Gas/op
$\mathbb{F}_q$ mul	2,314
$\mathbb{F}_q$ square	2,301
$\mathbb{F}_{q^3}$ mul	36,472
$\mathbb{F}_{q^3}$ square	26,813
$\mathbb{F}_{q^6}$ mul	166,774
$\mathbb{F}_{q^6}$ square	113,788

После финального аудита MNT6 verifier приведен к той же архитектуре, что и MNT4: используется общий multi-Miller аккумулятор и один квадрат на раунд. Из $r_{\text{MNT6}} = q - N$ следует:

$$F \cdot c^{N-q} = F \cdot c^{-r} = 1.$$

Таблица 9: Результаты MNT6

Режим	Gas
Miller loop, packed pointer blob	93,254,054
Packed Frobenius/w0 FE	38,428,108
Полное single pairing: Miller + packed FE	131,685,843
Диагностический residue digest одного pairing	103,277,505
Два Miller loop + полная FE для equation	226,078,963
Общий multi-Miller + c -проверка для equation	172,004,717

Экономия общего residue verifier относительно контрольного equation baseline:

$$226,078,963 - 172,004,717 = 54,074,246 \text{ gas},$$

то есть 23.92%.

7.3 Cycle-native accounting

Rust-модуль проверяет параметры цикла через arkworks, строит reference pairings и считает operation/constraints accounting для будущего native relation layer.

Таблица 10: Предварительная модель MNT-cycle-native relation

Сторона	Башня	Miller rounds	Additions	Prepared relation
MNT4	$\mathbb{F}_{q^2}/\mathbb{F}_{q^4}$	376	124	24,126
MNT6	$\mathbb{F}_{q^3}/\mathbb{F}_{q^6}$	376	123	48,942
Сумма строительных блоков				73,068

Для сравнения:

$$\text{BN254 non-native sparse Miller estimate} = 63,309,128,$$

$$\text{Sonobe/CycleFold-like anchor} \approx 9,000,000.$$

Число 73,068 нельзя называть стоимостью готового CycleFold. Это ручная модель multiplication constraints для подготовленных relation fragments. Она показывает порядок потенциального выигрыша нативной постановки и обосновывает направление следующего этапа.

Если механически сложить лучшие прямые EVM equation verifier-ы двух сторон, получится:

$$93,734,789 + 172,004,717 = 265,739,506 \text{ gas}.$$

Эта сумма иллюстрирует границу прямого on-chain исполнения. Будущий folding не должен выполнять оба verifier-а on-chain; его смысл как раз в переносе relation layer в нативный цикл.

8 Исследовательское направление lollipop-305

8.1 Идея

Статья *Cycles of supersingular elliptic curves for pairing-based proof systems* [2] предлагает конструкции, в которых часть арифметики выполняется над меньшими полями. Для исследовательского примера lollipop-305 выбран параметр:

$$x = 8004046504391788107635887004283725454478544674,$$

$$p = x^2 - x + 1, \quad q = x^2 + 1.$$

Оба основных поля имеют размер около 305 бит и помещаются в два слова EVM.

8.2 Кривые конструкции

Таблица 11: Pairing-компоненты lollipop-305

Часть	Кривая и поле	Целевое поле
stick	pairing-friendly prototype над $\mathbb{F}_p$	$\mathbb{F}_{p^4}$
cycle E	$E/\mathbb{F}_{p^2} : y^2 = x^3 + (\mu + 1)x$	$\mathbb{F}_{p^4}$
cycle $\hat{E}$	$\hat{E}/\mathbb{F}_{q^2} : y^2 = x^3 + (\eta + 2)$	$\mathbb{F}_{q^6}$

Проверены отношения:

$$\#E(\mathbb{F}_{p^2}) = p^2 + 1 = qN_q,$$

$$\#\hat{E}(\mathbb{F}_{q^2}) = q^2 - q + 1 = pN_p.$$

8.3 Оптимизированная 2-limb арифметика

Для двухсловного Montgomery-умножения generic lower bound содержит:

$$4 \text{ произведения для } ab + 4 \text{ произведения для REDC.}$$

Для lollipop-305 старший limb модуля меньше 2^{49} . Поэтому:

$$a_1b_1 < 2^{98} < 2^{256}.$$

Одно полное 512-битное восстановление можно заменить обычным MUL. Дополнительно убрана запись limb-a, который сразу удаляется CIOS-сдвигом.

8.4 Почему для третьей кривой нужны два аккумулятора

Для stick и E можно использовать Article640-style residue verifier. Для $\hat{E}/\mathbb{F}_{q^2}$ степень вложения равна 3. Стандартное удаление знаменателей линий не работает в требуемой форме. Поэтому backend и Solidity отдельно накапливают:

$$F_{\text{num}}, \quad F_{\text{den}}.$$

Таблица 12: Сравнение 2-limb lollipop-305 и 3-limb MNT4

Операция	lollipop-305	MNT4-753	Выигрыш
F_p mul	1,018	2,959	$2.91\times$
F_p square	940	2,947	$3.14\times$
F_{p^2} mul	4,171	11,877	$2.85\times$
F_{p^2} square	3,254	10,592	$3.26\times$
F_{p^4} mul	14,942	42,764	$2.86\times$
F_{p^4} square	13,211	36,606	$2.77\times$

На doubling-шаге:

$$F_{\text{num}} \leftarrow F_{\text{num}}^2 \ell_{\text{dbl}}(P),$$

$$F_{\text{den}} \leftarrow F_{\text{den}}^2 v_{\text{dbl}}(P).$$

На addition-шаге:

$$F_{\text{num}} \leftarrow F_{\text{num}} \ell_{\text{add}}(P),$$

$$F_{\text{den}} \leftarrow F_{\text{den}} v_{\text{add}}(P).$$

В конце проверяется отношение:

$$c^p F_{\text{den}} = F_{\text{num}}.$$

Это эквивалентно residue-проверке частного:

$$F = F_{\text{num}}/F_{\text{den}}.$$

Ведение двух аккумуляторов и 305-битное возведение c^p в $\mathbb{F}_{q^6}$ делают третью часть заметно дороже.

8.5 Результаты lollipop-305

Таблица 13: Pairing-verifier-ы lollipop-305

Часть	Режим	Function gas
stick	residue FE	8,671,771
cycle $E/\mathbb{F}_{p^2}$	residue FE	18,283,781
cycle $\hat{E}/\mathbb{F}_{q^2}$	Weil fallback	200,977,272
cycle $\hat{E}/\mathbb{F}_{q^2}$	prepared-Ate raw $F_{\text{num}}/F_{\text{den}}$	76,422,749
cycle $\hat{E}/\mathbb{F}_{q^2}$	prepared-Ate + residue	106,441,661
Полный lollipop-cycle	stick + E + улучшенный $\hat{E}$	133,397,213

Для fixed-cache режима дополнительно сравнивались calldata+commitment и code-shards:

$$135,916,985 \text{ gas} \text{ против } 133,651,154 \text{ gas}.$$

Code-shards экономят около 2.27 млн gas за счет исключения повторной передачи примерно 273 КБ подготовленных линий.

Lollipop-305 показывает важный положительный факт: уменьшение числа limb-ов действительно дает почти трехкратный выигрыш на базовой арифметике. Но этого недостаточно для автоматического выигрыша полного цикла: структура pairing, степень расширения и возможность удаления знаменателей не менее важны, чем размер поля.

9 Сводные результаты и trade-off

Таблица 14: Основные результаты работы

Подход	Что измеряется	Результат
Наивный Tate baseline	строгая нижняя экстраполяция	2.55 млрд gas
MNT4 full optimized on-chain	fixed- Q , линии строятся on-chain	258.8 млн gas
MNT4 prepared sparse	кэш линий подготовлен заранее	79.6 млн gas
MNT4 Article640 residue	fixed-shards equation с проверкой G_1	93.7 млн gas
MNT6 full equation	два Miller loop + packed FE	226.1 млн gas
MNT6 Article640 residue	общий multi-Miller accumulator	172.0 млн gas
KZG opening BN254	короткая on-chain opening-проверка	133 тыс. gas
KZG + MNT4 relation	BN254 non-native sparse Miller estimate	63.3 млн constraints
Merkle/ordinary-FRI	ожидаемая модельная стоимость	78.3 млн gas
Merkle/ordinary-FRI	размер доказательства	2.07 МБ
MNT-cycle-native relation	MNT4 + MNT6 fragments	73,068 операций
lollipop-305	полный улучшенный исследовательский цикл	133.4 млн gas

9.1 Три формы стоимости

Полученные результаты показывают не один, а три взаимосвязанных вида расходов.

On-chain arithmetic. Если считать MNT-сопряжение непосредственно в EVM, придется платить gas за трехсловную арифметику, расширения поля и сотни шагов цикла Миллера.

Off-chain constraints. Если перенести вычисление в proof над BN254, MNT-арифметика становится non-native. Gas on-chain уменьшается, но растет число constraints, размер witness и время работы prover-а.

Calldata и proof size. Если использовать нативные таблицы и Merkle/FRI, можно уменьшить зависимость от BN254 non-native arithmetic, но стоимость возвращается через Merkle-пути, раскрытия таблиц и calldata.

9.2 Экономическая интерпретация

Для иллюстрации примем:

$$\text{ETH price} = 3000 \text{ USD.}$$

Тогда:

$$\text{cost}_{USD} = \text{gas} \cdot \text{gas price}_{\text{gwei}} \cdot 10^{-9} \cdot 3000.$$

Таблица 15: Иллюстративная стоимость on-chain вызова

Режим	Gas	5 gwei	20 gwei
MNT4 prepared sparse	79,588,799	\$1,194	\$4,775
MNT4 Article640 residue	93,734,789	\$1,406	\$5,624
MNT6 Article640 residue	172,004,717	\$2,580	\$10,320
lollipop full research-cycle	133,397,213	\$2,001	\$8,004

Эти числа не являются прогнозом стоимости транзакции: цена ETH и gas price меняются. Таблица показывает порядок расходов.

10 Итоговые выводы

1. Реализована и измерена оптимизированная 753-битная арифметика MNT4-753/MNT6-753 в Solidity/Yul. Для MNT4 экспериментально обоснован выбор Montgomery/CIOS, Karatsuba, специализированных умножений на нерезиденты, sparse lines и packed hot path.
2. Построена оптимизационная лестница от наивного Tate baseline (2.55 млрд gas по строгой нижней экстраполяции) до prepared MNT4 пути (79.6 млн gas). Это показывает эффект оптимизаций и одновременно сохраняющуюся границу EVM.
3. Реализована идея residue-проверки по ePrint 2024/640. Для MNT4 финальная часть сокращается с 28.25 млн до 6.13 млн gas overhead. Для MNT6 общий multi-Miller residue verifier снижает equation baseline на 54.07 млн gas.
4. Исследована замена цикла Миллера полиномиальной проверкой. KZG уменьшает on-chain gas, но переносит стоимость в BN254 non-native constraints. Merkle/FRI избегает части этого переноса, но требует большого proof и calldata.
5. Реализован предварительный MNT4/MNT6 cycle-native слой. Его accounting дает 24,126 и 48,942 multiplication operations для подготовленных fragments. Это не готовый CycleFold, но обоснованный строительный блок будущего folding.
6. Исследована конструкция lollipop-305. Двухсловная арифметика действительно дешевле MNT4 примерно в 2.8–3.3 раза. Однако третья кривая требует двух аккумуляторов.

муляторов числителя и знаменателя, поэтому полный прямой EVM-цикл остается дорогим.

Главный вывод работы:

Без MNT-prescompile или нового proof-system слоя нельзя одновременно получить малую on-chain стоимость и малую off-chain стоимость простым переносом вычислений. Выполненная работа локализует причины этого ограничения, подтверждает их кодом и измерениями и показывает наиболее перспективное направление продолжения: MNT4/MNT6 cycle-native relation layer с последующим folding.

11 Воспроизводимость

Финальный очищенный проект расположен в каталоге:

```
/Users/a.i.semenov/diploma-final.
```

Основной запуск:

```
cd /Users/a.i.semenov/diploma-final  
./scripts/run_all.sh
```

Он выполняет gas-бенчмарки арифметики, MNT4/MNT6 Article640-модули, Rust fixture cross-check, MNT-cycle accounting и ordinary-FRI cost model.

Полный аудит завершен успешно:

- MNT6 Article640: 13/13 Foundry-тестов;
- Rust fixture MNT6 побайтно совпадает с tracked JSON;
- MNT-cycle accounting: 5/5 тестов;
- ordinary-FRI cost model воспроизводится;
- основной MNT6 verifier имеет runtime-размер 20,312 байт, что укладывается в EIP-170.

Список литературы

- [1] Andrija Novakovic, Liam Eagen. *On Proving Pairings*. Cryptology ePrint Archive, Paper 2024/640, 2024. URL: <https://eprint.iacr.org/2024/640.pdf>.
- [2] Craig Costello, Pratyush Korpai. *Cycles of supersingular elliptic curves for pairing-based proof systems*. Cryptology ePrint Archive, Paper 2024/1627, 2024. URL: <https://eprint.iacr.org/2024/1627.pdf>.
- [3] Privacy and Scaling Explorations. *Sonobe: a modular library for folding schemes*. URL: <https://github.com/privacy-ethereum/sonobe>.